

# M2S-09: Step 9 — Optimize: Validate, Optimize, and Decommission

**Series:** M2S — Managed to SaaS Migration | **Notebook:** 9 of 9 | **Phase:** Run | **Step:** Optimize | **Created:** March 2026 | **Last Updated:** 09/28/2026

The migration is functionally complete. Agents are reporting, configurations are applied, integrations are reconnected, SaaS-exclusive features are adopted, and users are trained. This final step closes the loop: validate that every success criterion is met, optimize the SaaS environment for long-term performance, obtain stakeholder sign-off, and decommission the Managed cluster.

**M2S Migration Journey — 3 Phases / 9 Steps**

**Plan:** 1. Discover | 2. Strategize | 3. Design

**Upgrade:** 4. Prepare | 5. Execute | 6. Integrate

**Run:** 7. Enable | 8. Expand | **9. Optimize**

## Table of Contents

- 1. [Migration Validation](#)
- 2. [Performance Optimization](#)
- 3. [Dynatrace Intelligence Baseline Establishment](#)
- 4. [Data Retention Optimization](#)
- 5. [Migration Success Metrics](#)
- 6. [Stakeholder Sign-Off](#)
- 7. [Managed Decommissioning](#)
- 8. [Series Conclusion](#)
- 9. [Step Completion Checklist](#)

---

## Optimization Use Cases

With the migration complete, focus optimization efforts on these SaaS-enabled use cases:

| Use Case                             | SaaS Capability                                        | Priority |
|--------------------------------------|--------------------------------------------------------|----------|
| Automated incident management        | Workflows + problem triggers                           | High     |
| Automated remediation / self-healing | Workflows + HTTP request actions                       | High     |
| Automated release validation         | Site Reliability Guardian                              | Medium   |
| Digital experience management        | RUM, Session Replay, Synthetic                         | High     |
| Application security management      | Runtime vulnerability detection, code-level protection | High     |
| Log management and analytics         | Grail + OpenPipeline                                   | High     |
| Business analytics                   | Business events + Grail                                | Medium   |

## Operational Procedure Review

Review and update operational procedures now that you're on SaaS:

| Procedure                       | Update Required                                                             |
|---------------------------------|-----------------------------------------------------------------------------|
| Release schedules               | Dynatrace SaaS updates bi-weekly automatically — no action needed           |
| Notifications and alerting      | Review thresholds now that Dynatrace Intelligence baselines are established |
| High availability               | SaaS provides 99.5%+ SLA — document this in your HA plans                   |
| Capacity planning               | No more cluster capacity planning — SaaS scales with licensing              |
| Decommission old infrastructure | Remove Managed cluster servers, old ActiveGates, old network rules          |

# Prerequisites

| Requirement                  | Details                                                                                     |
|------------------------------|---------------------------------------------------------------------------------------------|
| Steps 1-8 Complete           | All migration phases finished — Discovery through Enable                                    |
| SaaS Tenant Active           | All agents reporting, configurations applied, integrations connected                        |
| Entity Inventory from Step 1 | Original Managed entity counts for comparison                                               |
| Managed Environment Running  | Still accessible for comparison (not yet decommissioned)                                    |
| Stakeholder Access           | Ability to obtain sign-off from platform, security, application, and executive stakeholders |

## Order of Operations — You Are Here

This notebook covers operations **9-11** of the 11-step Order of Operations (see **M2S-02: Step 2 — Strategize**) defined in Step 2:

| #  | Operation                                   | Status                                                                                                  |
|----|---------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 1  | Assess — Inventory current environment      | Step 4: Prepare    |
| 2  | Provision — SaaS tenant and access (SSO)    | Step 4: Prepare    |
| 3  | Install — New ActiveGates in parallel       | Step 4: Prepare    |
| 4  | Migrate — Configuration and integrations    | Step 5: Execute    |
| 5  | Rebuild — Dashboards, zones, alerts         | Step 5: Execute    |
| 6  | Redirect — OneAgents to SaaS                | Step 5: Execute    |
| 7  | Reconnect — Integrations and extensions     | Step 6: Integrate  |
| 8  | Migrate — Remaining config and integrations | Step 6: Integrate  |
| 9  | <b>Validate — Data flow and performance</b> | <b>This notebook</b>                                                                                    |
| 10 | <b>Cutover — Full switch to SaaS</b>        | <b>This notebook</b>                                                                                    |

| #  | Operation                          | Status        |
|----|------------------------------------|---------------|
| 11 | Decommission — Managed environment | This notebook |

## 1. Migration Validation

Validation is the most critical activity in this step. A migration is not complete until every success criterion is verified with data. Compare current SaaS entity counts and data flows against the inventory captured during Step 1 (Discover).

### 1.1 Entity Coverage

Run each query below and compare the results against your Managed inventory from Step 1. Any discrepancy must be investigated before proceeding.

```
// Host count – compare to Step 1 inventory
fetch dt.entity.host
| summarize hostCount = count()

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | summarize hostCount = count()
```

```
// Service count – compare to Step 1 inventory
fetch dt.entity.service
| summarize serviceCount = count()

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | summarize serviceCount = count()
```

```

// Application count – compare to Step 1 inventory
smartscapeNodes "FRONTEND"
| filter frontend.type == "web"
| summarize appCount = count()

// Smartscape (preferred, verified 07/2026): dt.entity.application maps to the
FRONTEND node,
// filtered on frontend.type == "web" (mobile apps are the same node with
frontend.type ==
// "mobile"). This corrects an earlier note here that claimed no Smartscape
equivalent existed.
// FRONTEND also carries id_classic holding the APPLICATION-* id, for joining
migrated and
// unmigrated queries. Unlike ActiveGate, `fetch dt.entity.application` does
still work and remains
// a genuine fallback – but it reads the classic entity store, which can
retain entities Smartscape
// (live topology) no longer lists, so cross-check if the two counts disagree.
// Classic fallback: fetch dt.entity.application | summarize appCount =
count()

```

```

// Process group count – compare to Step 1 inventory
fetch dt.entity.process_group
| summarize processGroupCount = count()

// Do not substitute smartscapeNodes "PROCESS" here: it lists individual
processes
// (dt.entity.process_group_instance), not process GROUPS – a different grain
that inflates the
// count. Keep the classic count for this comparison.

```

```
// Synthetic monitor count by type – compare to Step 1 inventory
smartscapeNodes "BROWSER_MONITOR", "HTTP_MONITOR",
"NETWORK_AVAILABILITY_MONITOR"
| summarize syntheticCount = count(), by:{type}
| sort type asc

// Smartscape (preferred, verified 07/2026): dt.entity.synthetic_test maps to
the BROWSER_MONITOR
// node (individual steps are a separate BROWSER_MONITOR_STEP node). HTTP
monitors are HTTP_MONITOR,
// multi-protocol monitors NETWORK_AVAILABILITY_MONITOR, private locations
SYNTHETIC_LOCATION. This
// corrects an earlier note here that claimed no Smartscape equivalent
existed. Unlike ActiveGate,
// `fetch dt.entity.synthetic_test` does still work and remains a genuine
fallback – it reads the
// classic entity store, which can retain entities Smartscape (live topology)
no longer lists.
// Classic fallback (browser monitors only): fetch dt.entity.synthetic_test |
summarize syntheticCount = count()
```

## Entity Coverage Comparison

Fill in the table below with the results from the queries above and the inventory from Step 1:

| Entity Type        | Expected (Step 1 Inventory) | Actual (SaaS) | Status    |
|--------------------|-----------------------------|---------------|-----------|
| Hosts              | —                           | —             | [ ] Match |
| Services           | —                           | —             | [ ] Match |
| Applications       | —                           | —             | [ ] Match |
| Process Groups     | —                           | —             | [ ] Match |
| Synthetic Monitors | —                           | —             | [ ] Match |
| ActiveGates        | —                           | —             | [ ] Match |

**Note:** Small discrepancies in service or process group counts are expected — SaaS may discover additional services through improved auto-detection. Investigate any entity type where the SaaS count is *lower* than the Managed inventory.

## 1.2 Data Flow Validation

Verify that all three pillars of observability — metrics, logs, and traces — are flowing continuously into SaaS with no gaps.

```
// Metric continuity – count hosts with active CPU data
timeseries avgCpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd hasData = arrayAvg(avgCpu)
| filter isNotNull(hasData)
| summarize hostsWithData = count()
```

```
// Log ingestion continuity – volume per 5-minute bucket
fetch logs, from:-1h
| summarize logCount = count(), by:{time_bucket = bin(timestamp, 5m)}
| sort time_bucket desc
| limit 24
```

```
// Trace/span continuity – volume per 5-minute bucket
fetch spans, from:-1h
| summarize spanCount = count(), by:{time_bucket = bin(start_time, 5m)}
| sort time_bucket desc
| limit 12
```

```
// Log sources – verify all expected sources are present
fetch logs, from:-1h
| summarize count = count(), by:{log.source}
| sort count desc
| limit 10
```

## 1.3 Data Gaps Detection

Gaps in data flow indicate agents that lost connectivity during migration or misconfigured network routes. A healthy migration shows consistent counts across all time buckets. The query returns the host count for every 5-minute bucket; if `minHosts` is lower than `maxHosts`, hosts dropped out at some point — look at `hostsReporting` to see when.

```
// Count reporting hosts per 5-minute bucket – drops indicate gaps
// to:-5m leaves out the bucket still being filled, which would otherwise
always read as a drop
timeseries c = count(dt.host.cpu.usage), from:-6h, to:-5m, interval:5m, by:
{dt.entity.host}
| fieldsAdd reporting = iCollectArray(if(isNotNull(c[]), 1, else: 0))
| summarize hostsReporting = sum(reporting[])
| fieldsAdd minHosts = arrayMin(hostsReporting), maxHosts =
arrayMax(hostsReporting)
```

```
// Service request volume over time – look for unexpected drops
fetch spans, from:-6h
| filter span.kind == "server"
| makeTimeseries requestCount = count(), interval: 5m
```

**Tip:** Run these gap-detection queries across a 6-hour or 24-hour window. A single 5-minute bucket with zero data is acceptable during a maintenance window, but multiple consecutive empty buckets indicate a connectivity problem that must be resolved.

## 2. Performance Optimization

With data flowing and entities validated, optimize the SaaS environment for long-term operational efficiency. Focus on query performance, dashboard responsiveness, and alert signal quality.

### 2.1 Query Performance

Queries migrated from Managed may not follow SaaS best practices. Review and optimize all frequently-used queries:

| Optimization            | Technique                                                                                   | Impact                          |
|-------------------------|---------------------------------------------------------------------------------------------|---------------------------------|
| <b>Add time filters</b> | Always use <code>from:</code> on every <code>fetch</code> and <code>timeseries</code> query | Reduces scan scope dramatically |

| Optimization               | Technique                                                                              | Impact                                         |
|----------------------------|----------------------------------------------------------------------------------------|------------------------------------------------|
| <b>Filter early</b>        | Apply <code>filter</code> immediately after <code>fetch</code> , before any transforms | Reduces records processed in downstream stages |
| <b>Select fields early</b> | Use <code>fieldsKeep</code> or <code>fieldsRemove</code> to drop unneeded columns      | Reduces memory usage per record                |
| <b>Use aggregations</b>    | Replace raw-data queries with <code>summarize</code> or <code>makeTimeseries</code>    | Returns summary instead of millions of rows    |
| <b>Target buckets</b>      | Use <code>bucket:</code> parameter to limit scan to specific Grail buckets             | Avoids scanning irrelevant data                |
| <b>Limit results</b>       | Add <code>limit</code> as the final command                                            | Prevents oversized result sets                 |

## 2.2 Dashboard Optimization

Slow dashboards are the most visible symptom of unoptimized queries. Prioritize dashboards used by operations and executives:

| Issue             | Root Cause                             | Solution                                                           |
|-------------------|----------------------------------------|--------------------------------------------------------------------|
| Slow loading      | Too many tiles, unoptimized queries    | Reduce tile count to < 20; optimize each tile's DQL                |
| Tile timeouts     | Missing time filters, broad scan scope | Add <code>from:</code> and <code>bucket:</code> to every query     |
| Excessive data    | Raw data tiles instead of aggregations | Replace with <code>summarize</code> or <code>makeTimeseries</code> |
| Redundant queries | Multiple tiles querying the same data  | Use shared variables or combine into fewer queries                 |

## 2.3 Alert Tuning

Alert quality is critical in the first weeks after migration. Expect initial noise as Dynatrace Intelligence establishes baselines:

| Symptom         | Likely Cause                               | Action                                              |
|-----------------|--------------------------------------------|-----------------------------------------------------|
| Too many alerts | Thresholds too sensitive for SaaS baseline | Temporarily raise thresholds; revisit after 2 weeks |

| Symptom          | Likely Cause                                     | Action                                                        |
|------------------|--------------------------------------------------|---------------------------------------------------------------|
| Missing alerts   | Notification channels not fully reconnected      | Re-verify every channel from Step 6                           |
| Noisy alerts     | Transient issues during baseline period          | Add minimum duration conditions (e.g., 5 consecutive minutes) |
| Duplicate alerts | Overlapping alerting rules migrated from Managed | Consolidate into fewer, broader rules                         |

**Important:** Do NOT disable alerting during the baseline period. Instead, route non-critical alerts to a staging channel (e.g., a dedicated Slack channel) so the team can review without alert fatigue.

### 3. Dynatrace Intelligence Baseline Establishment

Dynatrace Intelligence uses machine learning to establish behavioral baselines for every entity in the environment. In a new SaaS environment, Dynatrace Intelligence starts with no historical data and must learn normal behavior from scratch.

#### Baseline Timeline

| Baseline Type     | Time Required | What Dynatrace Intelligence Learns  |
|-------------------|---------------|-------------------------------------|
| Response time     | 2-7 days      | Normal service response patterns    |
| Error rates       | 2-7 days      | Expected error levels per service   |
| Resource usage    | 7-14 days     | CPU, memory, disk patterns per host |
| Traffic patterns  | 7-14 days     | Daily and weekly traffic cycles     |
| Seasonal patterns | 4-6 weeks     | Monthly or periodic business cycles |

## What to Expect During Baseline Period

| Week      | Behavior                                                                               |
|-----------|----------------------------------------------------------------------------------------|
| Week 1    | Higher alert volume — Dynatrace Intelligence flags many deviations as it lacks history |
| Week 2    | Alert volume decreases — short-term baselines established                              |
| Weeks 3-4 | Near-normal alert volume — weekly patterns learned                                     |
| Weeks 5-6 | Stable — Dynatrace Intelligence has sufficient history for accurate anomaly detection  |

**Warning:** Expect more alerts in the first 1-2 weeks. This is normal and expected. Do NOT disable Dynatrace Intelligence or suppress problem detection. Instead, review each alert to confirm Dynatrace Intelligence is detecting real issues versus baseline noise. The learning period is essential for long-term accuracy.

## Monitor Detected Problem Trends

Track the problem trend over the first weeks post-migration. A healthy baseline establishment shows declining problem counts over time:

```
// Problem trend over last 7 days – expect a declining curve as baselines establish
fetch dt.davis.problems, from:-7d
| makeTimeseries problemCount = count(default: 0), interval: 24h
```

```
// Problem breakdown by category – identify which types are noisiest
fetch dt.davis.problems, from:-7d
| summarize count = count(), by:{event.category}
| sort count desc
```

```
// Mean time to resolve – track improvement over baseline period
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate ==
false
| makeTimeseries avgDurationHours = avg(resolved_problem_duration / 1h), time:
event.end
```

---

## 4. Data Retention Optimization

---

Grail provides flexible data retention through bucket configuration. Optimize retention to balance compliance requirements, operational needs, and cost.

### Retention Strategy by Data Type

| Data Type               | Recommended Retention | Rationale                                       |
|-------------------------|-----------------------|-------------------------------------------------|
| Logs (operational)      | 35 days               | Standard troubleshooting window                 |
| Logs (compliance/audit) | 90-365 days           | Regulatory requirements                         |
| Metrics                 | 5 years (aggregated)  | Built-in aggregated retention; no action needed |
| Spans/traces            | 35 days               | Sufficient for root cause analysis              |
| Business events         | 90-365 days           | Business intelligence and trend analysis        |
| detected problems       | 365 days              | Year-over-year reliability comparison           |

### Bucket Targeting for Query Performance

Use the `bucket:` parameter in queries to target specific Grail buckets and avoid scanning irrelevant data:

```
// Target specific log buckets instead of scanning all logs
fetch logs, bucket:{"app_logs_*"}, from:-1h
| filter loglevel == "ERROR"
| summarize count = count(), by:{log.source}
```

## Cost Optimization Tips

| Technique                                                                    | Impact                                            |
|------------------------------------------------------------------------------|---------------------------------------------------|
| Set shorter retention for high-volume, low-value data                        | Directly reduces storage costs                    |
| Route verbose debug logs to a separate bucket with 7-day retention           | Keeps debug data available without long-term cost |
| Use <code>samplingRatio:</code> for exploratory queries on large datasets    | Reduces query cost for ad-hoc analysis            |
| Archive compliance data to external storage if Grail retention exceeds needs | Avoids paying Grail rates for cold storage        |

## 5. Migration Success Metrics

Define and measure success quantitatively. Each metric has a target — the migration is complete when all targets are met.

| Metric                 | Target                           | Actual | Status |
|------------------------|----------------------------------|--------|--------|
| Host coverage          | 100% of Managed inventory        | —      | []     |
| Service coverage       | 100% of Managed inventory        | —      | []     |
| Application coverage   | 100% of Managed inventory        | —      | []     |
| Data gaps              | < 15 minutes total               | —      | []     |
| Alert delivery         | 100% of channels verified        | —      | []     |
| Dashboard availability | 100% of dashboards rendering     | —      | []     |
| Integration success    | 100% of integrations reconnected | —      | []     |
| User access            | 100% of users can log in via SSO | —      | []     |

**Note:** Do not proceed to stakeholder sign-off until every metric shows a passing status. Any failing metric must be resolved first.

## 6. Stakeholder Sign-Off

Formal sign-off marks the official end of the migration project. Each stakeholder group validates the aspects of the migration relevant to their responsibility.

### Sign-Off Matrix

| Stakeholder        | What They Validate                                                       | Approval | Date |
|--------------------|--------------------------------------------------------------------------|----------|------|
| Platform Team Lead | Entity coverage, data flow, alert delivery, dashboard availability       | [ ]      |      |
| Security Team      | SSO configuration, IAM policies, data masking, compliance certifications | [ ]      |      |
| Application Teams  | Service visibility, trace data, application-specific dashboards          | [ ]      |      |
| Operations Team    | Alert channels, runbook updates, incident workflow connectivity          | [ ]      |      |
| Executive Sponsor  | Overall migration success, budget, timeline adherence                    | [ ]      |      |

### Sign-Off Preparation

Before scheduling sign-off meetings, prepare the following:

1. **Migration summary report** — Entity counts, data flow validation results, success metrics
2. **Issue log** — Any open issues with resolution timeline
3. **Risk register** — Remaining risks (e.g., baselines still establishing)
4. **Decommission plan** — Timeline and steps for Managed shutdown
5. **Training completion report** — From Step 7 (Enable)

---

## 7. Managed Decommissioning

---

This is the point of no return. Decommissioning the Managed environment is the final act of the migration. Follow the order of operations carefully — rushing this step risks data loss and operational disruption.

### Order of Operations

These correspond to items 9-11 in the overall migration order of operations: Validate, Cutover, Decommission.

| Step | Action                                                                                                                      | Timing                             | Responsible         |
|------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------|---------------------|
| 1    | <b>Confirm no agents pointing to Managed</b> — Verify every OneAgent and ActiveGate is reporting to SaaS                    | Immediate                          | Platform Team       |
| 2    | <b>Keep Managed running for comparison</b> — Run both environments in parallel for a validation period                      | 2-4 weeks post-migration           | Platform Team       |
| 3    | <b>Export final Managed data and reports</b> — Download any data, dashboards, or reports not migrated                       | Before shutdown                    | Platform Team       |
| 4    | <b>Archive SaaS Upgrade Assistant deploy result CSVs</b> — These document what was migrated and any failures                | Before shutdown                    | Platform Team       |
| 5    | <b>Archive entity inventory and config mapping docs</b> — Preserve the Step 1 inventory and configuration mapping for audit | Before shutdown                    | Platform Team       |
| 6    | <b>Decommission Managed cluster</b> — Shut down Managed servers, release infrastructure                                     | After validation period + sign-off | Infrastructure Team |

# Pre-Decommission Verification

```
// Final host count in SaaS – this is the definitive count before Managed shutdown
fetch dt.entity.host
| summarize finalHostCount = count()

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | summarize finalHostCount = count()
```

```
// Final service count in SaaS
fetch dt.entity.service
| summarize finalServiceCount = count()

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | summarize finalServiceCount = count()
```

```
// Verify no data gaps in the last 24 hours before decommission
fetch logs, from:-24h
| summarize logCount = count(), by:{time_bucket = bin(timestamp, 1h)}
| sort time_bucket asc
```

# What to Archive Before Shutdown

| Artifact                                  | Format                  | Storage Location        |
|-------------------------------------------|-------------------------|-------------------------|
| SaaS Upgrade Assistant deploy result CSVs | CSV                     | Team SharePoint / Wiki  |
| Entity inventory from Step 1              | Spreadsheet             | Team SharePoint / Wiki  |
| Configuration mapping document            | Spreadsheet or Markdown | Git repository          |
| Managed dashboard screenshots             | PNG/PDF                 | Team SharePoint         |
| Migration timeline and decision log       | Document                | Project management tool |

| Artifact                     | Format       | Storage Location        |
|------------------------------|--------------|-------------------------|
| Stakeholder sign-off records | PDF or email | Project management tool |

## Post-Decommission Cleanup

After Managed is shut down:

| Task                      | Details                                                                               |
|---------------------------|---------------------------------------------------------------------------------------|
| Revoke Managed API tokens | Ensure no scripts still reference Managed endpoints                                   |
| Update DNS records        | Remove or redirect any DNS entries pointing to Managed cluster                        |
| Release infrastructure    | Return servers, release cloud resources, cancel hosting contracts                     |
| Update documentation      | Ensure all runbooks, architecture diagrams, and onboarding guides reference SaaS only |
| Notify stakeholders       | Send formal notification that Managed is decommissioned                               |

## 8. Series Conclusion

Congratulations on completing the M2S: Managed-to-SaaS Migration series. Here is a summary of what was accomplished across all nine steps:

| Step                 | Phase   | Achievement                                                                      |
|----------------------|---------|----------------------------------------------------------------------------------|
| <b>1. Discover</b>   | Plan    | Understood SaaS differences, inventoried Managed environment, assessed readiness |
| <b>2. Strategize</b> | Plan    | Defined migration approach, timeline, and risk mitigation strategy               |
| <b>3. Design</b>     | Plan    | Created target SaaS architecture, network topology, and IAM structure            |
| <b>4. Prepare</b>    | Upgrade | Provisioned SaaS tenant, deployed ActiveGates, configured SSO                    |

| Step                | Phase   | Achievement                                                                           |
|---------------------|---------|---------------------------------------------------------------------------------------|
| <b>5. Execute</b>   | Upgrade | Migrated configurations via SaaS Upgrade Assistant, migrated OneAgents                |
| <b>6. Integrate</b> | Upgrade | Reconnected dashboards, alerting, CI/CD, ITSM, extensions, and API scripts            |
| <b>7. Enable</b>    | Run     | Trained users, updated documentation, established support processes                   |
| <b>8. Expand</b>    | Run     | Adopted SaaS-exclusive capabilities — Grail, Notebooks, Workflows, Dynatrace Assist   |
| <b>9. Optimize</b>  | Run     | Validated migration, optimized performance, obtained sign-off, decommissioned Managed |

## Ongoing Success

The migration is complete, but the journey continues. Here are the key activities for the first 90 days post-migration:

| Timeframe         | Focus                                                                         |
|-------------------|-------------------------------------------------------------------------------|
| <b>Weeks 1-2</b>  | Monitor Dynatrace Intelligence baseline establishment; review and tune alerts |
| <b>Weeks 3-4</b>  | Optimize dashboards and queries; resolve any remaining integration issues     |
| <b>Weeks 5-8</b>  | Deepen adoption of SaaS-exclusive features; expand automation workflows       |
| <b>Weeks 9-12</b> | Conduct first quarterly review; measure ROI against pre-migration baseline    |

## Additional Resources

- [SaaS Upgrade Assistant Documentation](#)
  - [Upgrading from Dynatrace Managed to SaaS](#)
  - [Dynatrace Documentation](#)
  - [Dynatrace Community](#)
  - [Dynatrace University](#)
  - [Dynatrace Community: Upgrade to SaaS](#)
-

## 9. Step Completion Checklist

---

The migration is complete when every item below is confirmed.

| Checkpoint                                                               | Status                   |
|--------------------------------------------------------------------------|--------------------------|
| Entity counts match inventory from Step 1                                | <input type="checkbox"/> |
| No data gaps > 15 minutes in metrics, logs, or traces                    | <input type="checkbox"/> |
| All alerts delivering correctly to all channels                          | <input type="checkbox"/> |
| All dashboards showing data with acceptable performance                  | <input type="checkbox"/> |
| Dynatrace Intelligence baselines establishing (2+ weeks of data)         | <input type="checkbox"/> |
| Query and dashboard performance optimized                                | <input type="checkbox"/> |
| Data retention configured per compliance requirements                    | <input type="checkbox"/> |
| Stakeholder sign-off obtained from all parties                           | <input type="checkbox"/> |
| Managed environment running for comparison period (2-4 weeks)            | <input type="checkbox"/> |
| Migration artifacts archived (deploy results, inventories, mapping docs) | <input type="checkbox"/> |
| Managed environment decommissioned                                       | <input type="checkbox"/> |

---

## Summary

---

In Step 9, you:

- Validated migration completeness by comparing SaaS entity counts against the Step 1 inventory
- Verified continuous data flow across metrics, logs, and traces with no gaps
- Optimized query performance, dashboards, and alert configuration for SaaS
- Understood the Dynatrace Intelligence baseline establishment timeline and what to expect
- Configured data retention to balance compliance, operational needs, and cost
- Measured migration success against quantitative targets

- Obtained formal stakeholder sign-off
- Executed the Managed decommissioning order of operations
- Archived all migration artifacts for future reference

**Key Takeaway:** A migration is only complete when every success criterion is validated with data, every stakeholder has signed off, and the Managed environment is safely decommissioned. The DQL queries in this notebook provide the evidence needed to close the project with confidence.

---

*Thank you for completing the M2S: Managed-to-SaaS Migration Best Practices series!*

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*